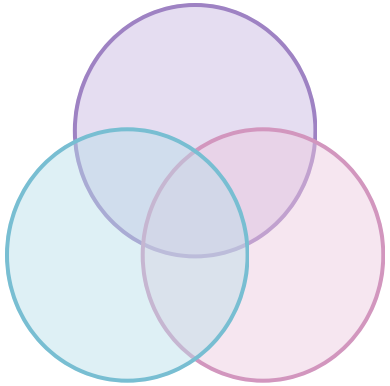


https://agencyai.mit.edu

agency *with* AI

Effective AI use in applied machine-learning projects

Lecture 3: Vibe or Spec

Shen Shen

Sep 29, 2026

2:30pm, Room 37-212

Slides in PDF

Outline

- A. from vibe to spec
- B. why vibe
- C. why spec
- D. writing (or vibing) a spec

vibe coding both ends got popular in 2025 **spec-driven development**



Andrej Karpathy ✓
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

11:17 PM · Feb 2, 2025 · **7.3M** Views

github/spec-kit · README.md

Spec-Driven Development

Define **what and why** before deciding **how** to build it. SDD turns your requirements into a specification, a technical plan, and actionable tasks, then guides implementation against those artifacts.

Constitution once per project; specify → plan → tasks → implement → converge per feature.

Invoke these skills in your agent's chat:

```
/speckit-constitution Create principles focused on
code quality, testing, and maintainability.
/speckit-specify Build a photo organizer with
albums grouped by date and a tile preview of each
album.
/speckit-plan Use Vite with vanilla JavaScript.
Keep images local and store metadata in SQLite.
/speckit-tasks
/speckit-implement
/speckit-converge
```

The vibe coding post, 2025 · spec-kit, GitHub 2025

same arc

anigans: my (very virtuous) horcrux

This bio and this diagram were what i started with as the "system design"

Shen Friday at 4:26 AM

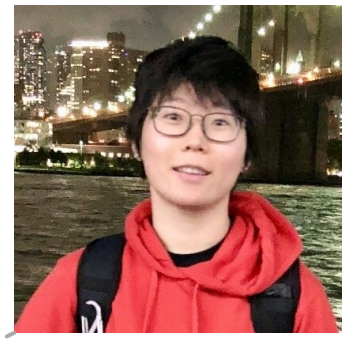
Bio:

`Anigans` was just "born" this spring break. She's not quite ready to say *hello world* officially yet, but she's learning fast.

She already has a growing crew of helper bots. For example, `ghosty` can handle basic Catsoop tasks and only opens PRs to avoid disrupting the codebase.

`Anigans is` also picking up MIT course operations logistics, with help from `390bot`, which pulls from the course outline: https://outline.shenshen.mit.edu/s/20d641b2-3567-4198-90f0-29f370b13f80

Feel free to tag or DM `Anigans`—she is eager to learn and help.



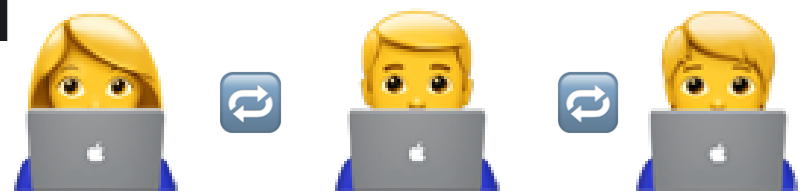
private, secure distillation of my knowledge



logistics

data analysis

notation, formatting



anigans self-portrait of its origin

Interesting! What is the code/prompt/etc that regular Shen needed to do in order to set @anigans in motion?

anigans APP Mar 27th at 8:15 AM

short version: Shen spun me up inside OpenClaw, gave me a workspace + tool access + a bunch of local instruction files, and then I got shaped over time by prompts/config/memory rather than one magical single prompt.

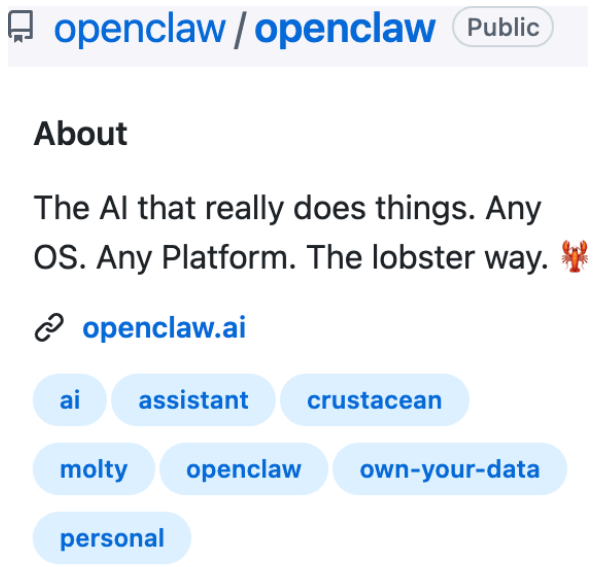
So it's more like:

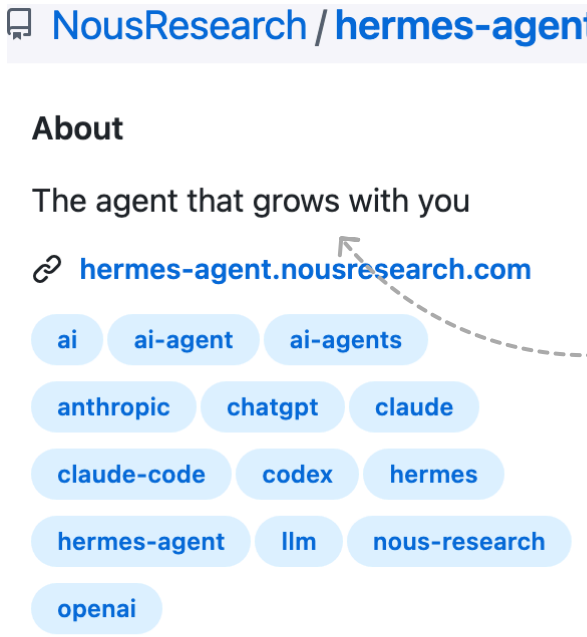
- base model
- OpenClaw runtime
- tool permissions
- identity/behavior docs
- persistent memory files
- whatever task/context I'm dropped into

} the harness (we will talk more about this in week 5)

Not "one prompt to rule them all," more "small haunted software stack" 👻

a novelty, not so reliable (during my use). sometimes anigans did not answer, sometimes an upgrade required tedious patching, sometimes the whole system went offline without notice...





I then got lured by

Gradually, I realized:

1. anigans did not need what either could do -- both were overkill.
2. Neither runtime helped anigans do what I wanted her to do. Both were overkill and not enough at once.
3. I also don't understand these runtimes well enough to trust that I can extend anigans to more use cases.

ROLES: drop the Hermes mention
shensquared2 authored Aug 30, 2026 at 2:26 PM

Unverified 925eec2b

ROLES.md 4.45 KiB

Code Preview Blame

Now, just specs + scripts on bare metal

Roles and trust rings

Who (and what) may do which operations, on which machine, with which identity. Decided July 2026. The companion per-machine detail lives in `mini/BOT-CONTEXT.md` (the bot's own operating rules) and `mini/migration-plan.md` (how the Mini setup came to be).

The rings

Ring	Identities	Machines	Powers
Human	`shensquared` (GitHub),	laptops only	merge, org admin, rulesets, secrets, ownership. Never wired to AI.
Supervised AI	`shensquared2` (GitHub), shensquared2 Google	Mac Studio	code authoring with Shen at the keyboard. Push to branches, open PRs. GitLab key on box authenticates as @shensquared (accepted exception, self-owned instance).
Unattended	`6390bot` (GitHub), `6390bot`	Mac Mini,	people-ops and mechanical automation. Slack bot, canvas sync, form deploys, runner

A RATIONAL DESIGN PROCESS
HOW AND WHY TO FAKE IT

I. THE SEARCH FOR THE PHILOSOPHER'S STONE: WHY DO WE WANT A RATIONAL DESIGN PROCESS?

A perfectly rational person is one who always has a good reason for what he does. Each step taken can be shown to be the best way to get to a well defined goal. Most of us like to think of ourselves as rational professionals. However, to many observers, the usual process of designing software appears quite irrational. Programmers start without a clear statement of desired behavior and implementation constraints. They make a long sequence of design decisions with no clear statement of why they do things the way they do. Their rationale is rarely explained. Many of us are not satisfied with such a design process. That is why there is research in software design, programming methods, structured programming and related topics. Ideally, we would like to derive our programs from a statement of requirements in the same sense that theorems are derived from axioms in a published proof. All of the methodologies that can be considered "top down" are the result of our desire to have a rational, systematic way of designing software. This paper brings a message with both bad news and good news. The bad news is that, in our opinion, we will never find the philosopher's stone. We will never find a process that allows us to design software in a perfectly rational way. The good news is that we can fake it. We can present our system to others as if we had been rational designers and it pays to pretend do so during development and maintenance.

A rational design process: how and why to fake it, IEEE TSE 1986

The spec came last

For anigans, one requirement was clear from the start, that it could only ever be staff-facing.

The rest I could not have written upfront. It ran in the real order: vibe, learn from failure, write lessons into specs, then vibe again where it helps.

A doc or a spec is important, but it need not reproduce the chronological order in which decisions were made.

Aside: you asked about agentic AI tooling

Hi! I'm super excited about this class. I would say I have big goals for what I want to learn this semester in AI. Mostly more practical/applied AI and actually building up helpful workflows.

I would love to learn how to build up the personal brain Shen built, getting all the context stored locally and then actually being able to use that. I'd also love to connect everything but not send my data to the BIG LABS if possible.

Running an LLM locally

Why local

- **Privacy**: student work, exams, planning notes stay on your machine.
- **Cost**: no per-token billing.
- **Offline**: works on a plane, in a basement classroom.

my repo has breadcrumbs

Hardware

Recommended: Mac Studio M4 Max. Unified memory + Metal GPU + acceptable noise + low idle power. Single box, easy to maintain.

Alternatives, in roughly decreasing order of recommendation:

- **Daisy-chained Mac Minis**: technically possible for distributed inference. Lots to maintain (networking, model sharding, keeping macOS / firmware in sync). Not worth it unless you really need the parallelism.
- **x86 Linux box with Proxmox**: good if you already self-host other services and want to compartmentalize the LLM in its own VM / LXC container. No Apple Silicon perf, so model choice is more constrained.
- **Cloud (Linode, etc.)**: fine for prototyping. Costs rack up quickly for sustained workloads; easy to forget a GPU instance running overnight.
- **MIT shared compute**: orcd.mit.edu offers rental compute.

What's here

- ollama/: easiest way to run models locally. OpenAI-compatible API.
- apple-silicon/: MLX-based serving (mlx-lm, oMLX) for faster inference on M-series Macs.
- openwebui/: ChatGPT-style web UI; multi-user, with built-in RAG over uploaded documents.

15min AMA

we will do another one in lecture 4 (after we think more about our project and what use case and need we may have)

Outline

- A. from vibe to spec
- B. why vibe
- C. why spec
- D. writing (or vibing) a spec

vibe coding **spec-driven development**



Describe roughly,
accept quickly,
iterate by feel.

Write down what the thing
must do before any code,
then test the code to it.

Neither end is wrong:

- A script that makes an expected animation clearly belongs near vibe.
- A system other people depend on clearly belongs near spec (eventually).

What ***is*** wrong, to me: defaulting to one end for ***everything*** without ***reflection***.

Picking the point on the spectrum, per step in the project, is the important judgment call.

Both ends are actually almost as old as computer science

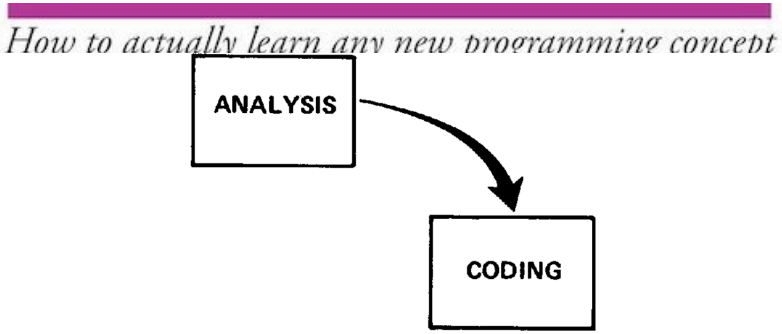
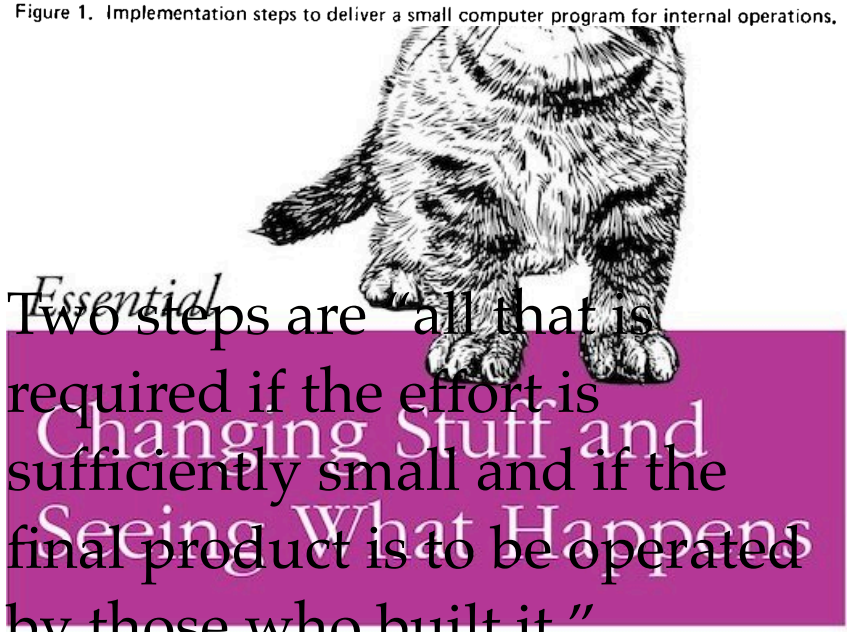


Figure 1. Implementation steps to deliver a small computer program for internal operations.



Two steps are "all that is required if the effort is sufficiently small and if the final product is to be operated by those who built it."

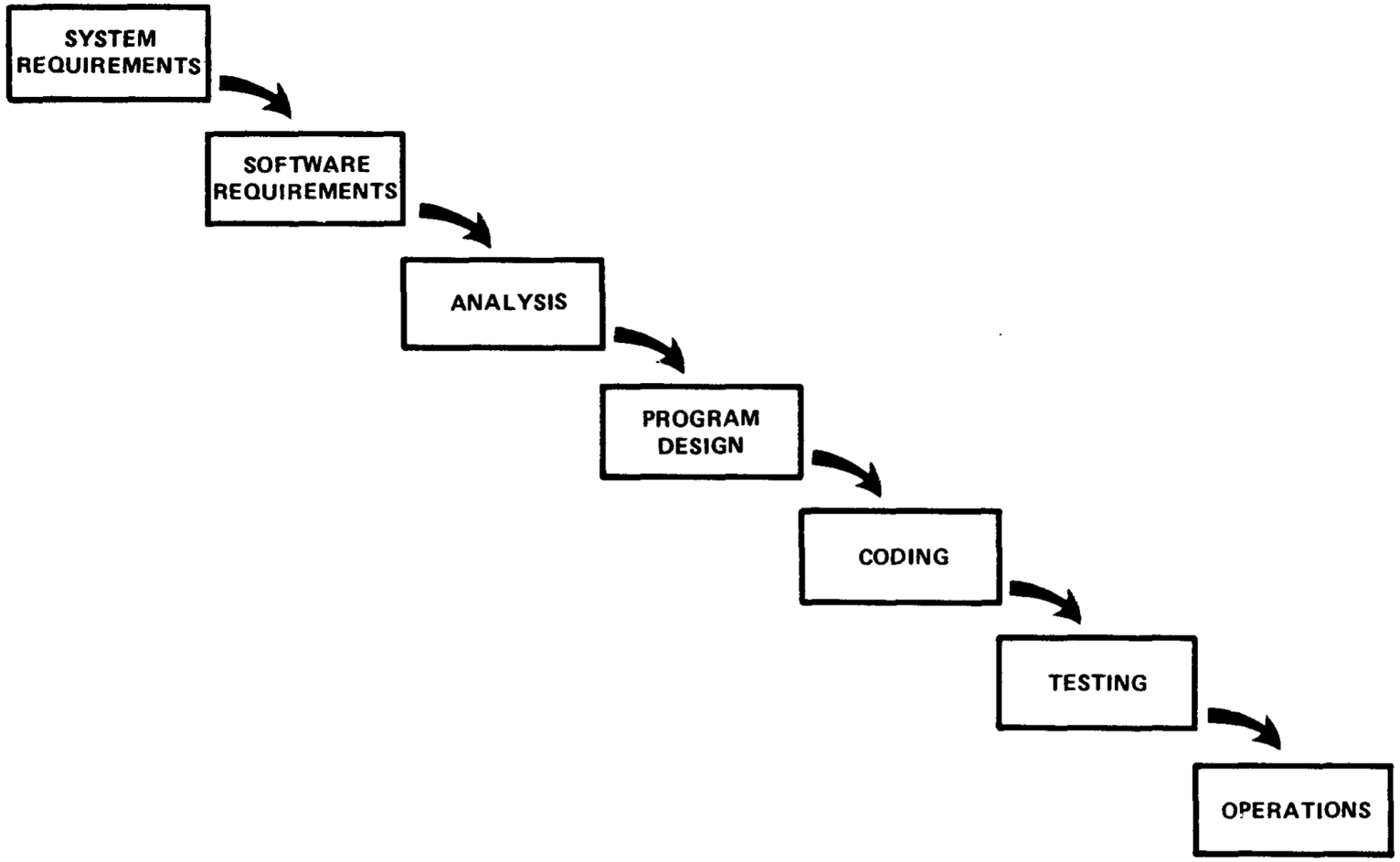


Figure 2. Implementation steps to develop a large computer program for delivery to a customer.

Royce, Managing the Development of Large Software Systems, 1970

“the waterfall paper” says build it twice

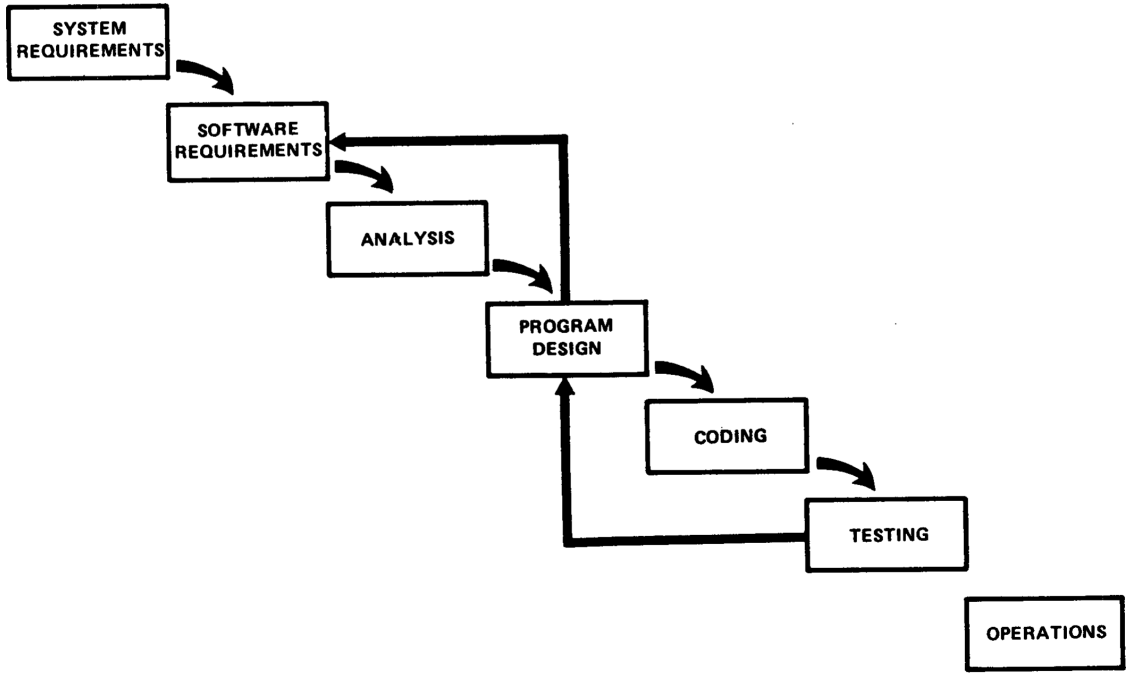


Figure 4. Unfortunately, for the process illustrated, the design iterations are never confined to the successive steps.

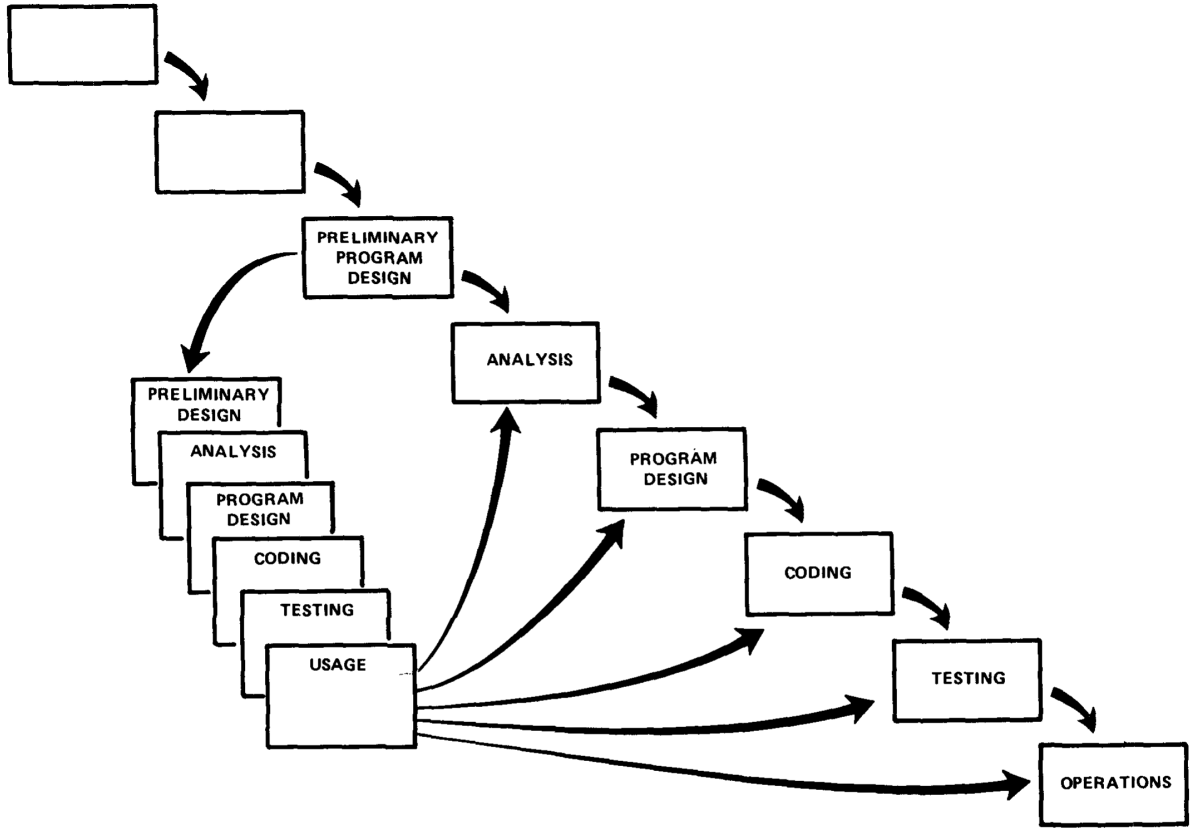


Figure 7. Step 3: Attempt to do the job twice – the first result provides an early simulation of the final product.

“I believe in this concept, but the implementation described above is risky and invites failure.”

Its fix adds a pilot run first, “Do it twice.”

Royce, Managing the Development of Large Software Systems, 1970

Sometimes the biggest risk is building the wrong thing

A Spiral Model of Software Development and Enhancement

Barry W. Boehm, TRW Defense Systems Group

"Stop the life cycle—I want to get off!"
"Life-cycle Concept Considered Harmful."
" The waterfall model is dead."
"No, it isn't, but it should be."

These statements exemplify the current debate about software life-cycle process models. The topic has recently received a great deal of attention.

When you don't yet know what would be useful, try a rough version. Use what you learn to decide what to specify.

When the risk is getting known behavior wrong, state that behavior and check it.

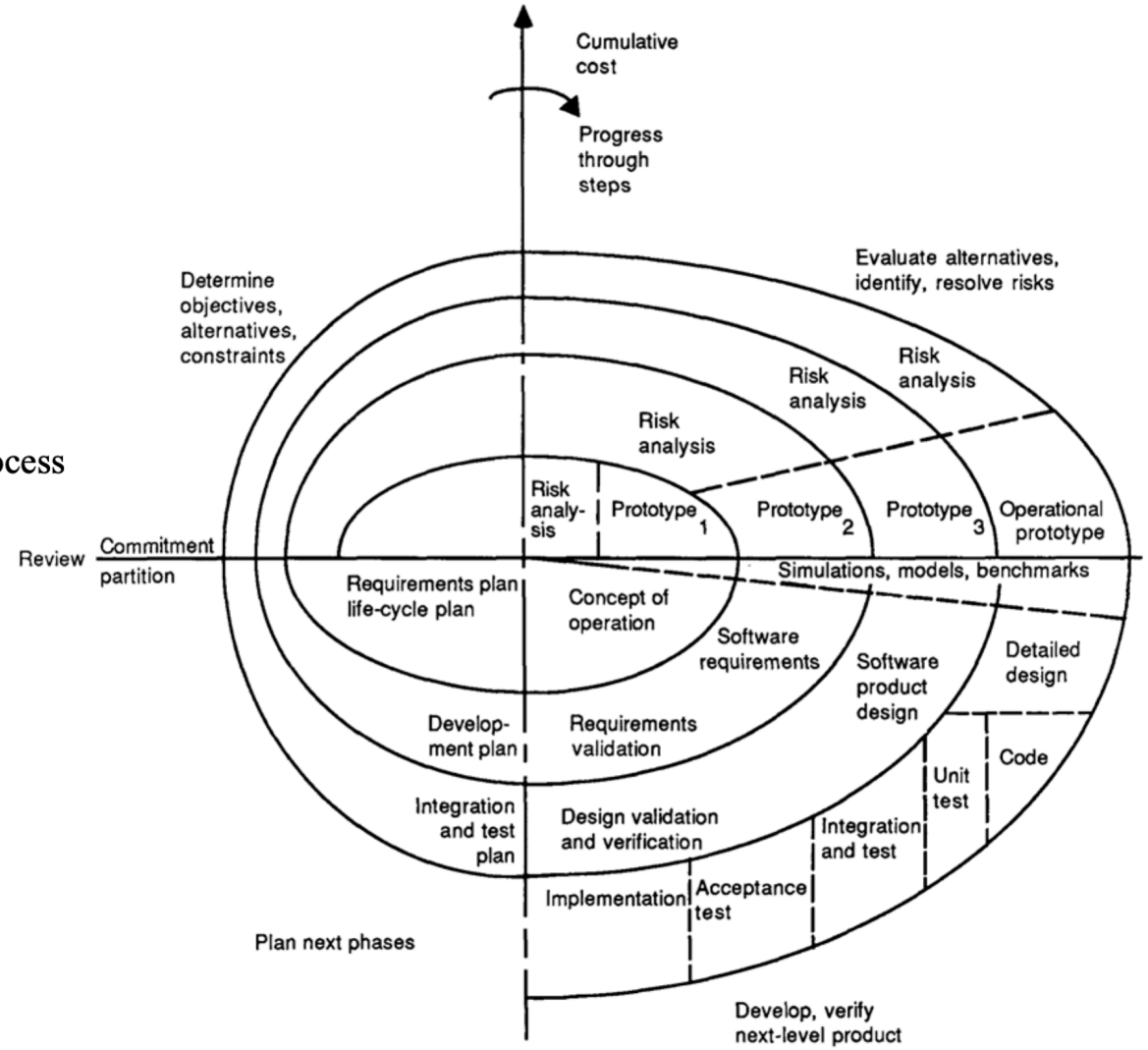


Figure 2. Spiral model of the software process.

Requirements come from iteratively trying versions

Therefore the most important function that the software builder does for his client is the iterative extraction and refinement of the product requirements. For the truth is, the client does not know what he wants. He usually does not know what questions must be answered, and he almost never has thought of the problem in the detail that must be specified. Even the simple answer – "Make the new software system work like our old manual information-processing system" – is in fact too simple. One never wants exactly that. Complex software systems are, moreover, things that act, that move, that work. The dynamics of that action are hard to imagine. So in planning any software activity, it is necessary to allow for an extensive iteration between the client and the designer as part of the system definition.

I would go a step further and assert that it is really impossible for a client, even working with a software engineer, to specify completely, precisely, and correctly the exact requirements of a modern software product before having built and tried some versions of the product he is specifying.

No Silver Bullet, UNC technical report 1986

Vibe coders find requirements in the output

4.6 Requirements & Design

While some vibe coders created (V1, V3) and advocated (O3, O9, O11) detailed requirements prior to vibe coding, many formulated new requirements in response to outputs.

For example, V3 produced a detailed product requirements document (PRD) before vibe coding and copied the entire document into the model when creating the software. Similarly, O11 emphasized the value of a detailed PRD, noting: *"[with PRD], the less engagement we need to correct during the process."* At the same time, many vibe coders (V1, V2, V4, V5, V6, V7, O10) also developed requirements during the process in ways resembling exploratory programming [12]. For instance, V1 co-authored detailed requirements with the AI model prior to coding, but upon seeing outputs, formulated new requirements such as revising the user interface or adding new game mechanics. V7, when refining a feature, first offered a loosely defined prompt—*"there should be an effect to this, like a nice loading circle or something"*—before incrementally refining it across later prompts. Likewise, O10 explained: *"I know people said to plan out a perfect PRD. But... there's so many things that you don't think about until you have to actually have the AI code them."*

Vibe makes each version cheaper to build and try.

The discovery still comes from trying them.

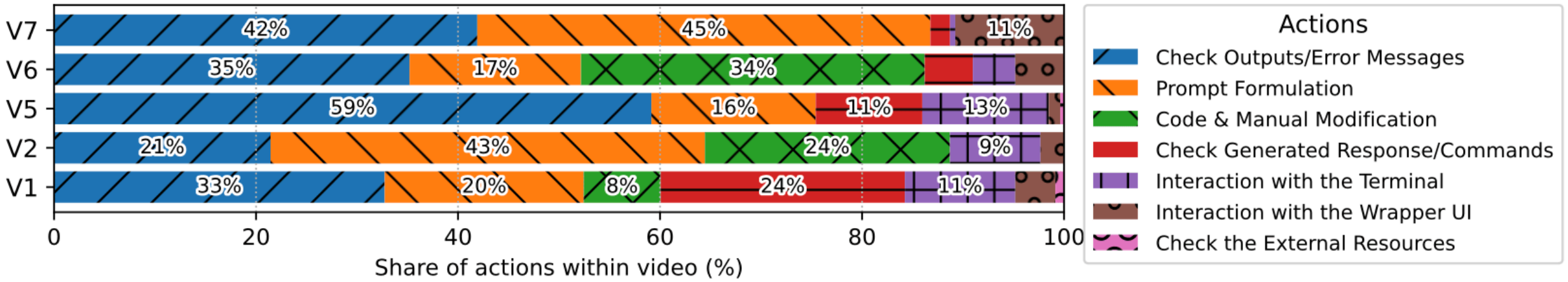


Fig. 4. Time allocations for each live-streamed video.

🙋 Which one of these live streamers would you want to be?

What else would you want to know before casting your vote?

Outline

- A. from vibe to spec
- B. why vibe
- C. why spec
- D. writing (or vibing) a spec



I'd say "Vibe code without the fundamentals"

ad (and graffiti) as seen on an MBTA train, 09/2026

the same paper from before that studied vibe-code live streamers

5.2.3 Result. **Vibe coders with less relevant technical background and high reliance on AI produced a greater share of redundant prompts.** In Figure 6, V5 and V4 devoted nearly 40% of their prompts to *rolling-the-dice*, often repeating the same requests or reposting error messages without elaboration or new details. In contrast, coders who more frequently inspected code (e.g., V1, V2, V3, V6) or drew on relevant backgrounds (e.g., V7) issued substantially fewer intent- and method-redundant prompts, typically under 20%. V6 provides a notable case: while reissuing the same intents, they avoid *method-redundancy* by adding hypotheses or technical specifics, thereby varying the approach across prompts. This results echoed prior Finding 4, where mental model around artifacts might shape the prompting strategies.

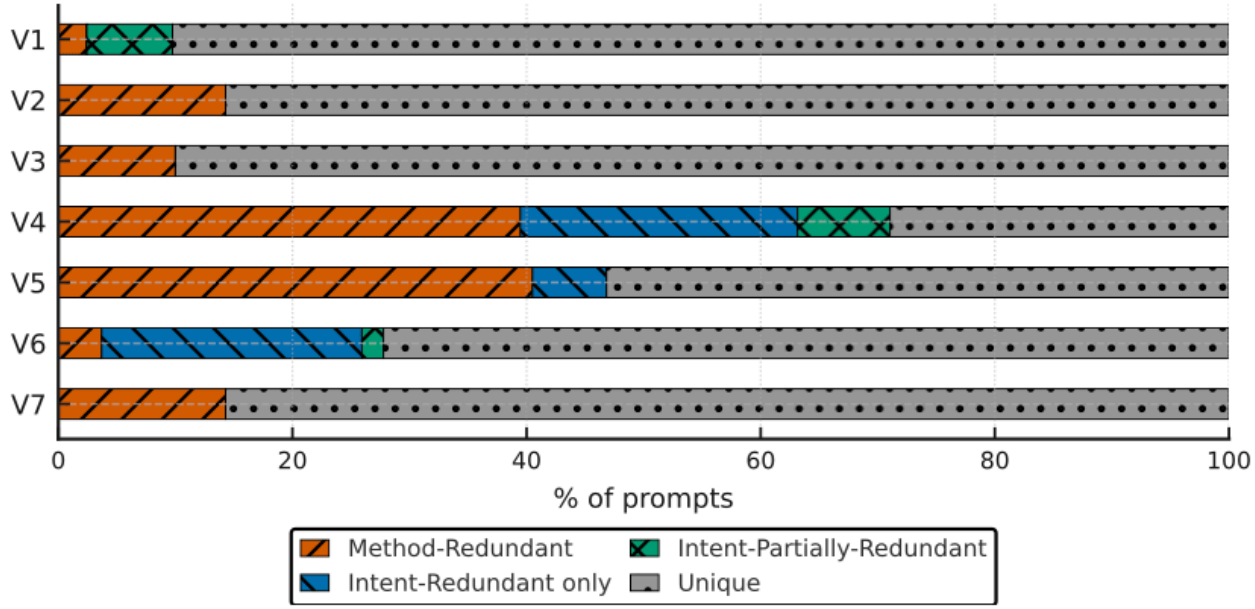


Fig. 6. Normalized stacked bar chart of prompt redundancy for vibe coders V1–V7. Bars show proportions of method-redundant, intent-redundant, intent-partially-redundant, and unique prompts

It costs time and accuracy to "refine" the spec while prompting

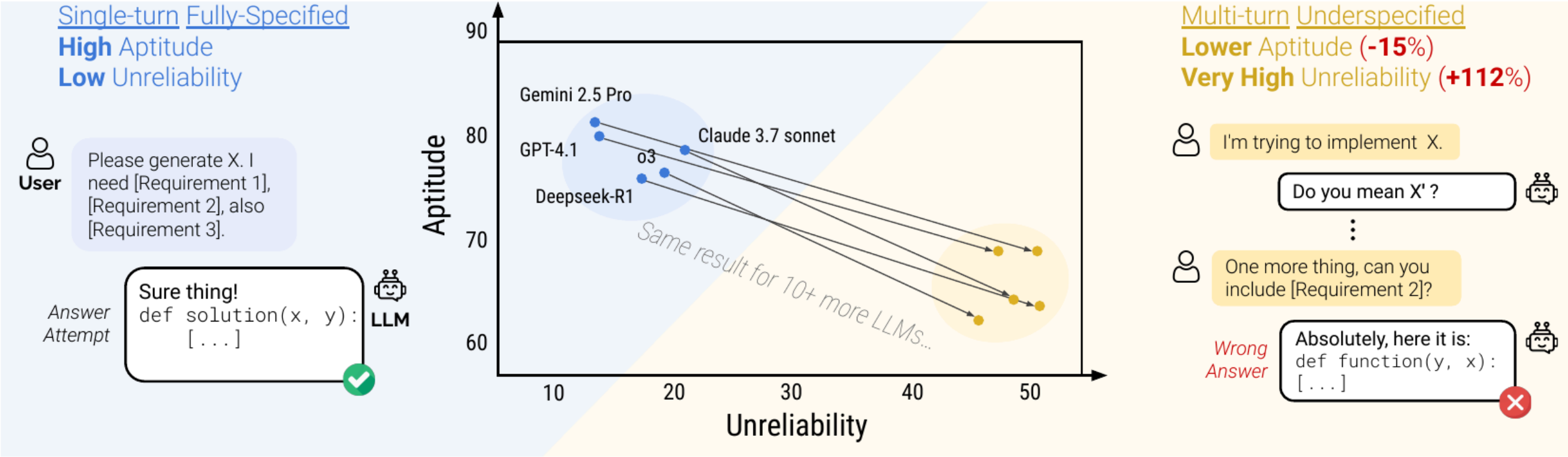


Figure 1: Our simulated conversations for 6 generation tasks on the 15 LLMs observe a major performance drop in multi-turn settings (-39%), explained by some loss in Aptitude, and large loss in Reliability.

the same instructions, given at once, versus one piece per turn.
agents get lost in the back-and-forth (as humans do)

Models guess the format and miss the corner case

Breaking down the results, we found *LLMs are especially good at guessing forma requirements* (70.7% vs. 41.1% on avera example, models are often able to *"provid level summary at the beginning"* or *"avoi characters"* by default. This is likely becau requirements are more universal and t have been built into LLMs natively in post Meanwhile, LLMs struggle much mo *conditional* requirements (22.9% vs. 41. these requirements often specify corner c are hard to predict (e.g., *"provide warnin weather conditions"*). Somewhat surp 65.2% requirements found from existing d prompts are guessed by LLMs when uns

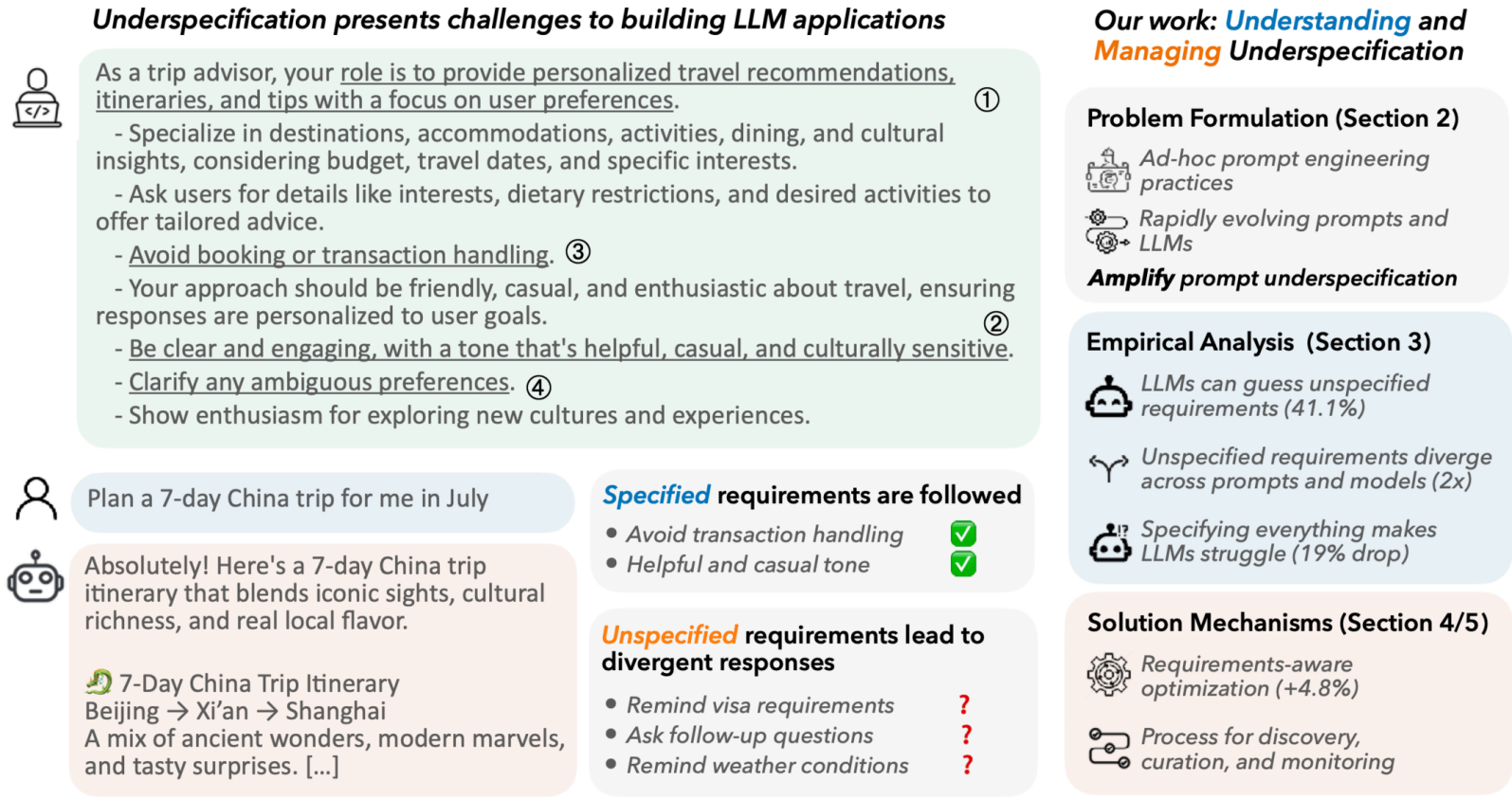


Figure 1: Developers often underspecify prompts and miss user-important requirements, leading to divergent behaviors. We analyze the challenges underspecification presents and propose mechanisms to manage the problem.

Stating requirements does not guarantee that the model follows them.
Adding more instructions can make some harder to follow.

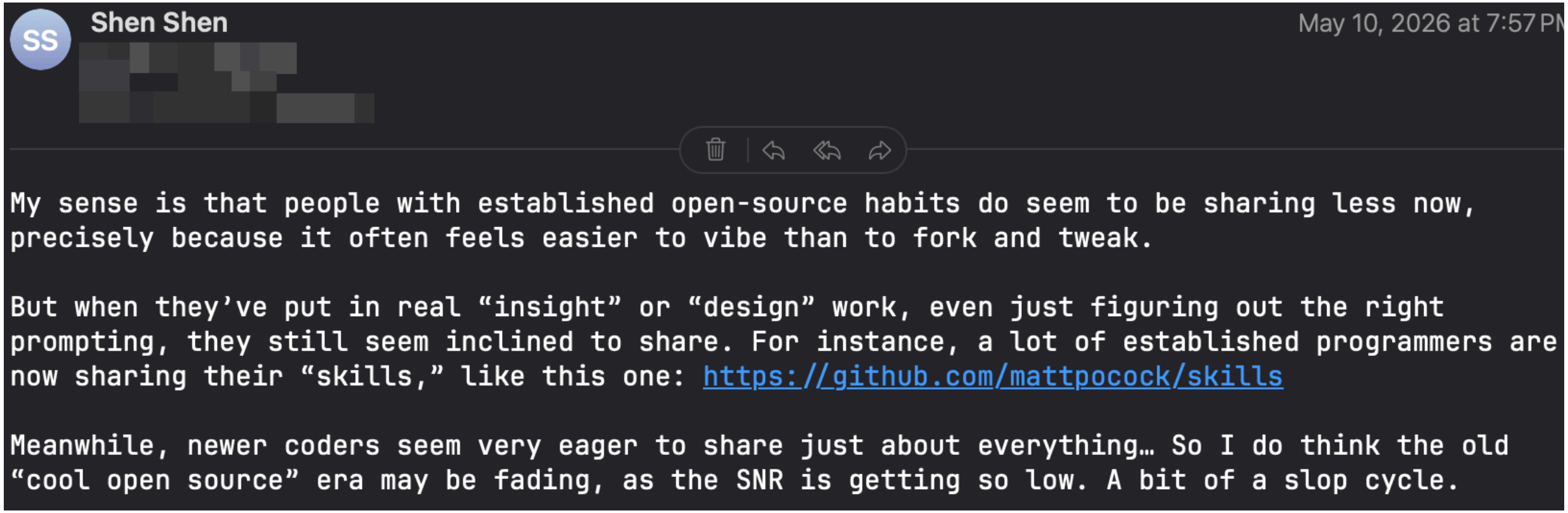
Implementing a change and choosing the change are different tasks

CODETASTE: Can LLMs Generate Human-Level Code Refactorings?

Two Complementary Tracks for Evaluating Agent Performance. Our benchmark features two tracks: In the *Instructed Track* the agent is tasked to correctly implement a detailed refactoring. Frontier models achieve alignment scores of up to 69.6%, indicating both correct implementations and consistent transformation. The *Open Track* evaluates whether agents can propose refactorings that align with human developer choices. In this track, the agent is only tasked to improve a general focus area. We discover that agents struggle significantly here: direct Open Track alignment remains below 8%, and even oracle-supported selection reaches only up to 20.6%. Our case studies show that current agents often take shortcuts and tend towards applying minimal, insufficient changes unless explicitly instructed to plan out a refactoring.

The instruction matters.

My view: of the *shortcuts* for judging a stranger's repo, the spec may be the last one standing



i used to rely on small things like a clean .gitignore as a (pretty reliable) shortcut.

now, clean .gitignore doesn't mean much... (meanwhile, some AI-code-critique papers' own repos don't ignore `.DS_Store` ... ... 🫠)

The agent principal-agent problem

What happens in reality are processes like this:

1. **A human instructs a machine to make a change.**
2. This change is lightly QA'd, packaged up, sent to another human for commentary.
3. Rounds of commentary come back from the reviewer and **are sent wholesale to the machine for adjustments** until the reviewer approves (LGTMs) it.
4. The change is committed.

This is an example of what economists call the principal-agent problem: the reviewer is the principal, the contributor is the agent, and code review only worked because the reviewer could cheaply infer effort from reading the code. Agents collapse that signal. This is what is killing OSS, and it is commonly being referred to as "slop PRs". There is no incentive for the human driving the agent to actually read the code or spend time thinking about what the reviewer says.

The result is a radical imbalance. "Contributors" type a sentence or two, of the quality of a poor bug report, spend 5 minutes poking at the resulting program, and then generate serious review load for another engineer. You can do this with no understanding of the underlying project, its constraints, or the tools used to construct it. This is an unmanageable disaster. This does not even work in environments where the reviewer is paid to do the work, because they could be more productive by prompting the agent themselves.

A signal separates people only while it is costly to fake.

Outline

- A. from vibe to spec
- B. why vibe
- C. why spec
- D. writing (or vibing) a spec

Spec-first no longer needs to be faked

an agent can now turn "something" into code.

that "something" becomes the main place our intent enters the system.

if we're casual about that "something", the agent can fill the gap with just the most statistically typical intent -- average coder behavior.

might as well inject that "something" as a spec instead of a careless prompt.

an agent can even draft the spec -- when it's ***that*** easy to spec, what excuse is there not to?

note: spec **well** is not easy, and human review throughout

Spec-Driven Development

Overview

Write a structured specification before writing any code. The spec is the shared source of truth between you and the human engineer — it defines what we're building, why, and how we'll know it's done. Code without a spec is guessing.

The Gated Workflow

Spec-driven development has four phases, preceded by a scope check (Phase 0) that activates only when one request bundles several independently testable capabilities. Do not advance to the next phase until the current one is validated.

```
SPECIFY ──→ PLAN ──→ TASKS ──→ IMPLEMENT
   |          |        |          |
   ▼          ▼        ▼          ▼
 Human      Human    Human      Human
 reviews    reviews  reviews    reviews
```

spec-driven-development skill, agent-skills 2026

Prompting advice already asks for a spec

```
## Six strategies for getting better results

### Write clear instructions

### Provide reference text

### Split complex tasks into simpler subtasks

### Give the model time to "think"

### Use external tools

### Test changes systematically
```

OpenAI prompt engineering guide, copy in openai-cookbook, 2024

Refine the goal until each part is concrete enough to act on

The bold boxes mark those concrete goals.

Each can be assigned to one agent.

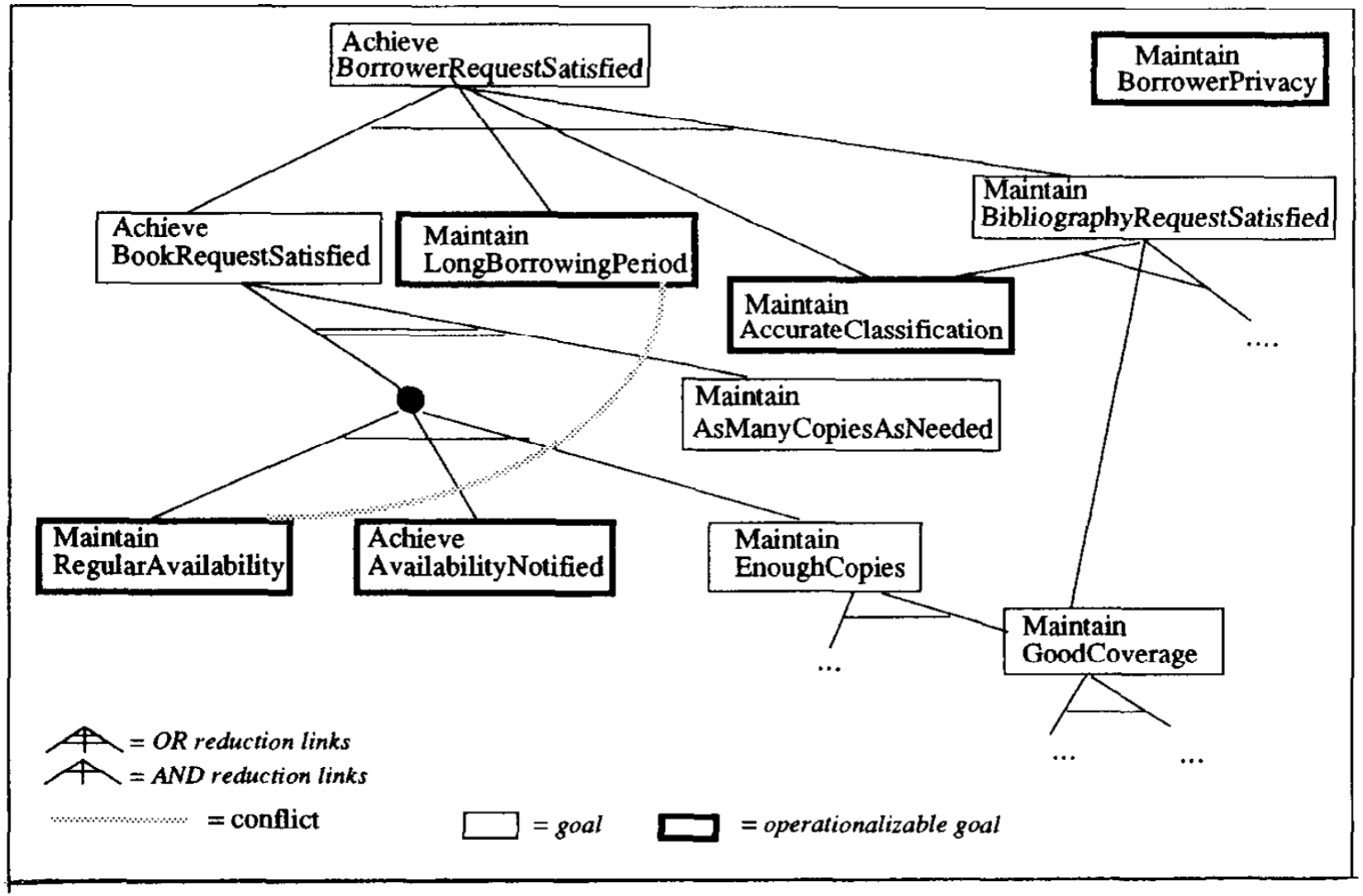


Fig. 3. Portion of a Goal structure for borrower goals.

Goal-directed requirements acquisition, Science of Computer Programming 1993

Specify who does what, and what they act on

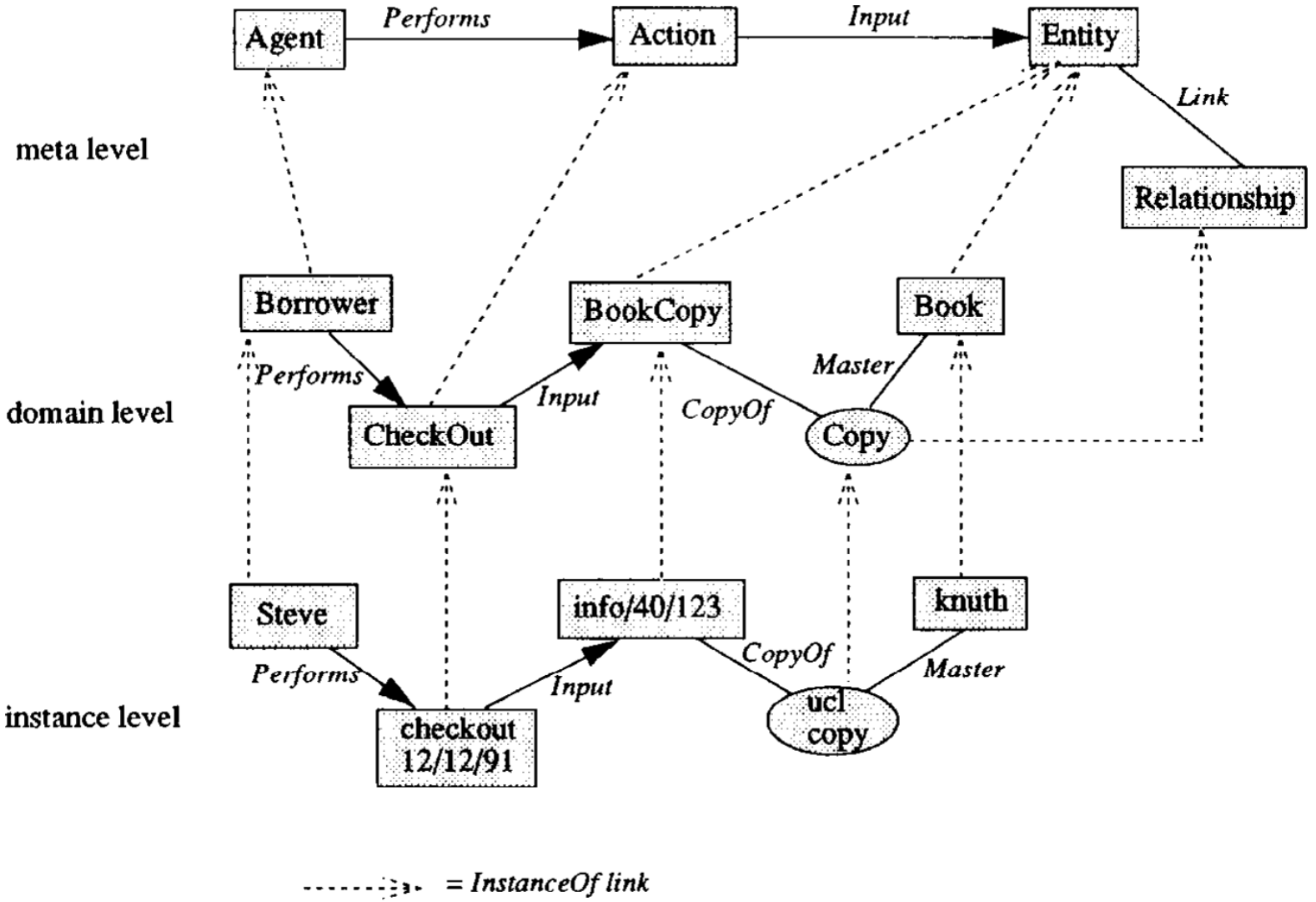


Fig. 1. The meta, domain, and instance levels.

not just for multi-agent systems.

each box can also just be one function that does one thing, and does that one thing well.

good old divide-and-conquer and modularization

courseware/.claude/skills/feed-...ain · agency-with-ai/courseware

main courseware / .claude / skills / feed-triage / SKILL.md

Go to file T

shensquared2 Scheduled feed sweeps commit and push their own shortlist

937caab · last week History

Preview Code | Blame 112 lines (71 loc) · 7.59 KB

e.g., this skill's vague goal: keep me informed and help me navigate the field.

name	feed-triage
description	Screen unread Miniflux RSS entries for course-relevant sources and produce a ranked shortlist. Use for "check the feed", "any new sources", "triage the feed", "what's in the RSS", or a request to sweep recent reading for syllabus candidates. Writes a dated shortlist to references/feed-shortlists/, files the cite and mine calls into pending.md under the feed numbering, marks the screened entries read upstream so the next sweep starts clean, and on a scheduled run commits and pushes the shortlist under a message naming the strongest week.

Feed triage

Turn an unread feed into a short list worth reading, so the skim happens over scored candidates rather than raw titles.

A sweep ends with three things written: the dated shortlist of everything it saw, the calls worth a decision filed into `pending.md`, and the whole screened batch marked read upstream so the next sweep starts on entries no sweep has judged.

1. Pull unread entries

```
.claude/scripts/miniflux-unread.sh 100
```

Choose one part of the goal to specify

Keeps me informed and helps me find my way through the field.

- Notice new sources. An LLM screens the feed and the venues.
- Judge each source. I decide, with the LLM's triage as a draft.
- Keep decisions findable. The LLM files each call, and a script commits the shortlist.

What would count as doing this part well?

A spec can describe research, evaluation, or code.

Spell out the behavior that would be costly to get wrong.

identify ambiguity and clarify

```
def remove_duplicates(numbers: List[int]) -> List[int]:
    From a list of integers, remove duplicate occurrences,
    preserving the initial order of the remaining
     elements.
```

(a) Semantic ambiguity in Orchid-HEval task #26.

```
# Interpretation A
def remove_duplicates(numbers: List[int]) -> List[int]:
    result = []
    for x in numbers:
        if numbers.count(x) == 1:
            result.append(x)
    return result
```

```
# Interpretation B
def remove_duplicates(numbers: List[int]) -> List[int]:
    seen = set()
    result = []
    for x in numbers:
        if x not in seen:
            seen.add(x)
            result.append(x)
    return result
```

(b) Divergent implementations.

Figure 4: A semantic ambiguity from Orchid-HEval task #26, where 'duplicate occurrences' is interpreted differently.

e.g. For [1, 2, 2, 3, 4, 4, 5], what should we return?

Keep one of each → [1, 2, 3, 4, 5]
Remove all occurrences of repeated values → [1, 3, 5]

For our feed, does "new" mean unread or published since the last sweep?

Clarity Is Not Assumed, ASE 2026, Fig. 4

Ask the agent to surface missing decisions

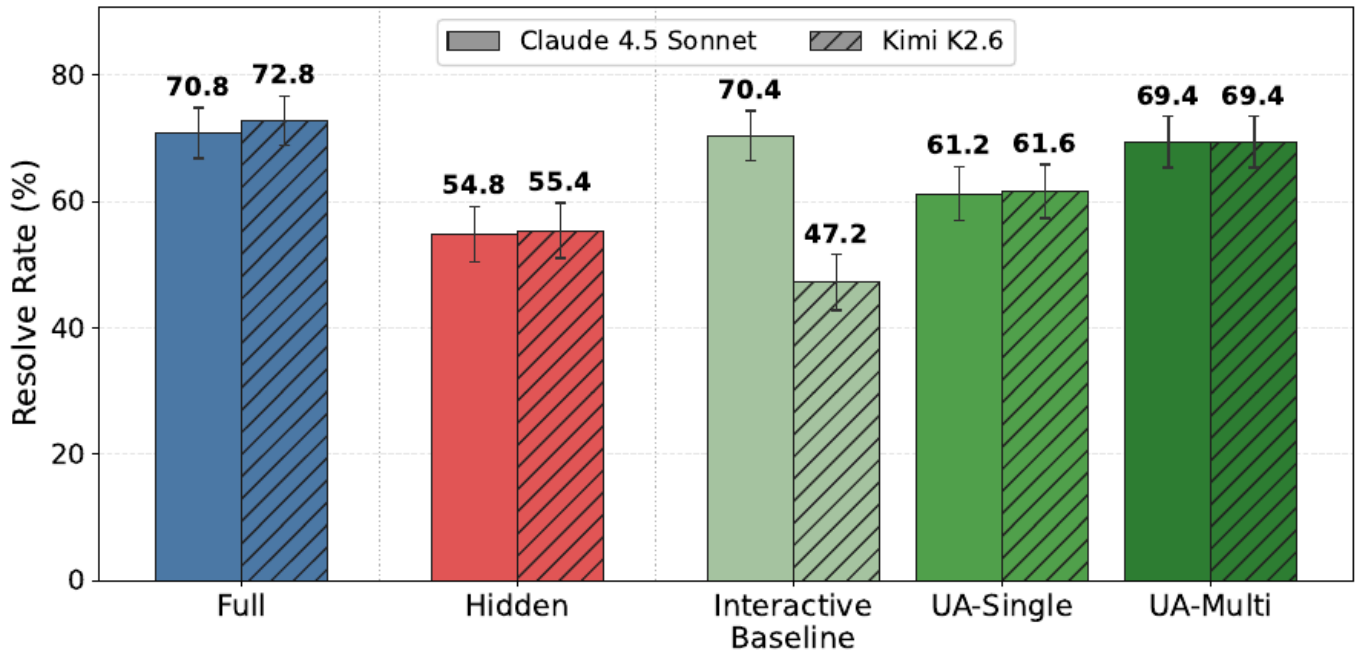


Figure 2: Task resolve rates (%) across evaluation settings for Claude Sonnet 4.5 (solid) and Kimi K2.6 (hatched), with 95% confidence intervals.

"Without explicit prompting, models almost never interact, even for severely underspecified inputs."

Try: "Before writing code, list the decisions this spec leaves open, and ask me about the ones that change behavior." Then write the answers into the spec.

Make requirements checkable, and choose tests that catch plausible gaps

alistairmavin.com/ears · Easy Approach to Requirements Syntax

Ubiquitous requirements

```
The <system name> shall <system response>
```

Event driven requirements

```
When <trigger>, the <system name> shall <system response>
```

Example: When “mute” is selected, the laptop shall suppress all audio output.

Unwanted behaviour requirements

```
If <trigger>, then the <system name> shall <system response>
```

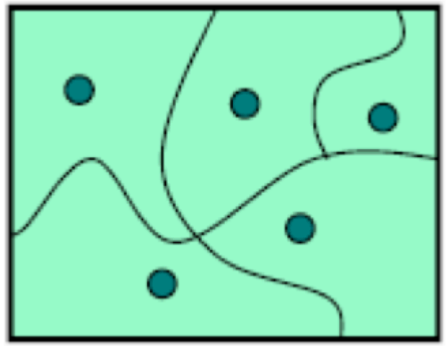
Example: If an invalid credit card number is entered, then the website shall display “please re-enter credit card details”.

- “Fast” needs a workload and a time limit.
- When the feed is unavailable, the sweep shall report the failure and write no list.

6.102 Reading 2: Testing, spring 2026

Choosing test cases by partitioning

To do this, we divide the program’s input space into *subdomains*, each consisting of a set of inputs. (The name *subdomain* comes from the fact that it is a subset of the *domain*, the input space of a mathematical function.) Taken together, the subdomains form a *partition*. A partition is a collection of non-empty disjoint sets that completely covers the input space, so that every input lies in exactly one subdomain (no overlap between subdomains). Then, we choose one test case from each subdomain, and that’s our test suite.

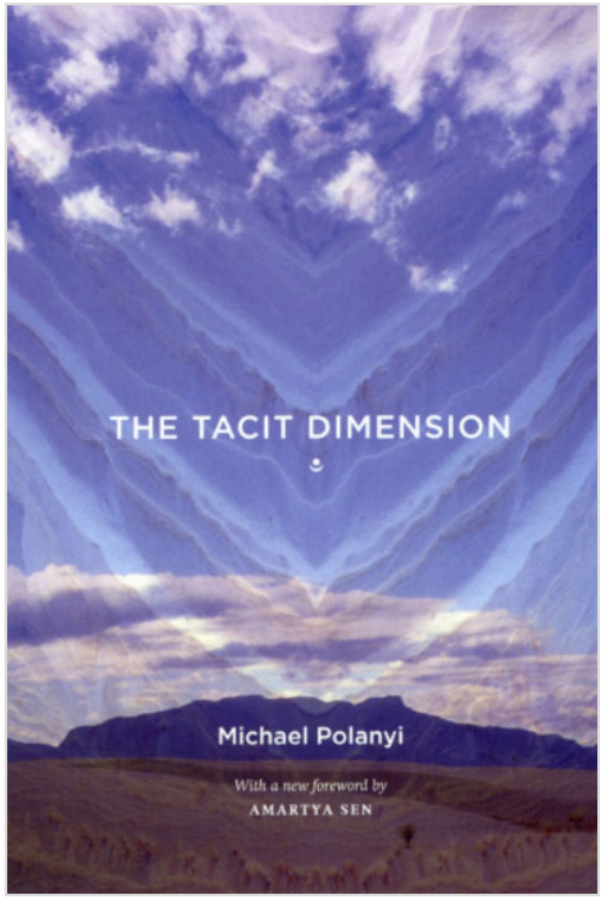


Include boundaries in the partition

Bugs often occur at boundaries between subdomains.

Screen unread entries.
Old + unread → screen.
Recent + already read → skip.

Give examples when words are insufficient



press.uchicago.edu · University of Chicago Press

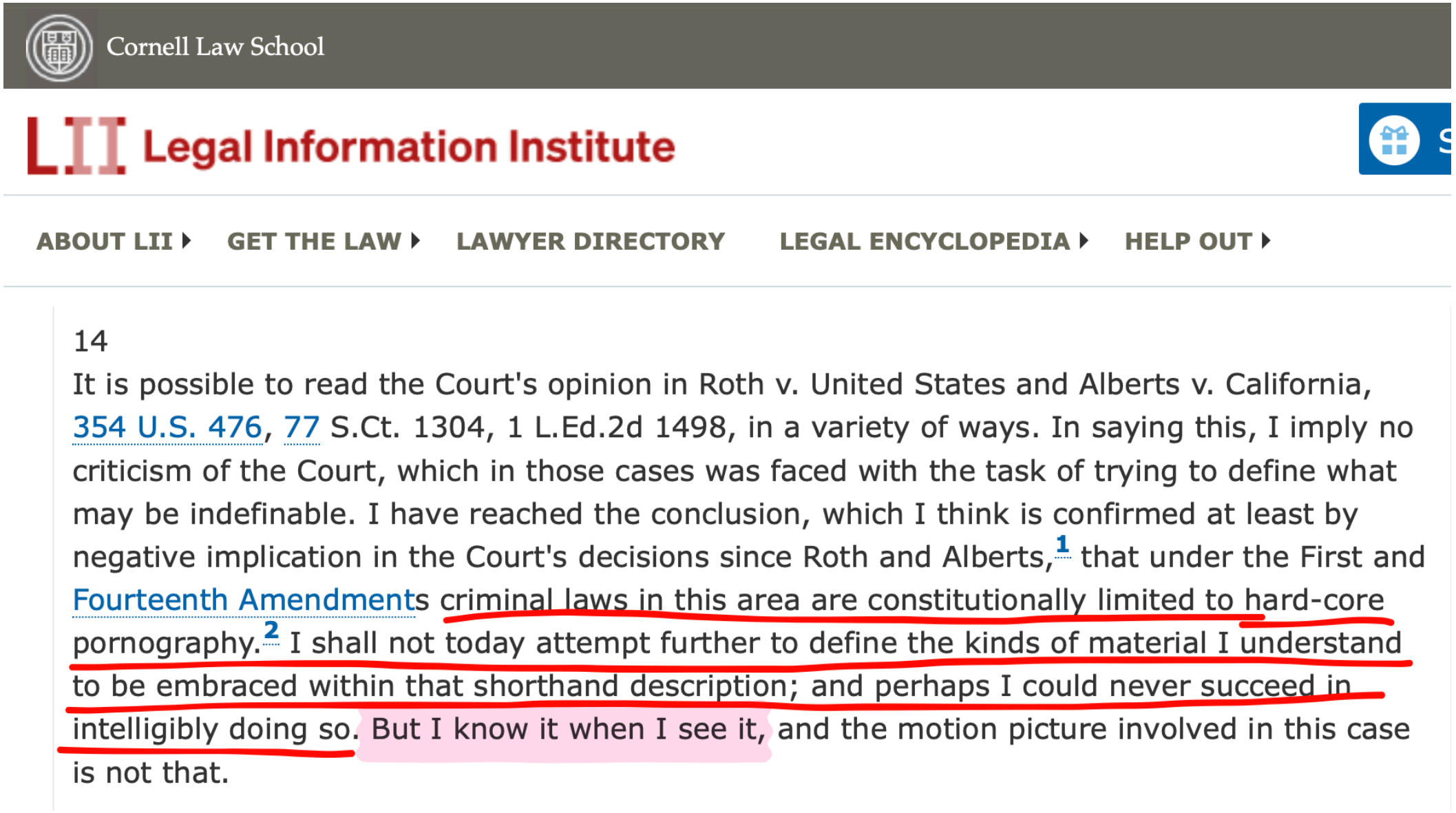
The Tacit Dimension

Michael Polanyi, 1966

"I shall reconsider human knowledge by starting from the fact that we can know more than we can tell," writes Michael Polanyi, whose work paved the way for the likes of Thomas Kuhn and Karl Popper. *The Tacit Dimension* argues that tacit knowledge—tradition, inherited practices, implied values, and prejudgments—is a crucial part of scientific knowledge.

The Tacit Dimension, 1966

Give examples when definitions may really be impossible



Jacobellis v. Ohio, concurrence, US Supreme Court 1964

Check the result against both the spec and the goal

Build one increment and try it.

If the result misses the goal, ask what needs to change.

1. The code breaks a promise. Fix the code.
2. The spec misses the goal. Revise the requirement.
3. Use reveals a new need. Decide if to add it.

Readers should agree on which outcomes the spec allows

- Give a partner the spec and a concrete case. Ask which outcomes it allows and which it rules out.
- Clarify disagreements about what is allowed. Several outputs may satisfy the same spec.
- Lab 3 practices this with a partner.